Bonita Project

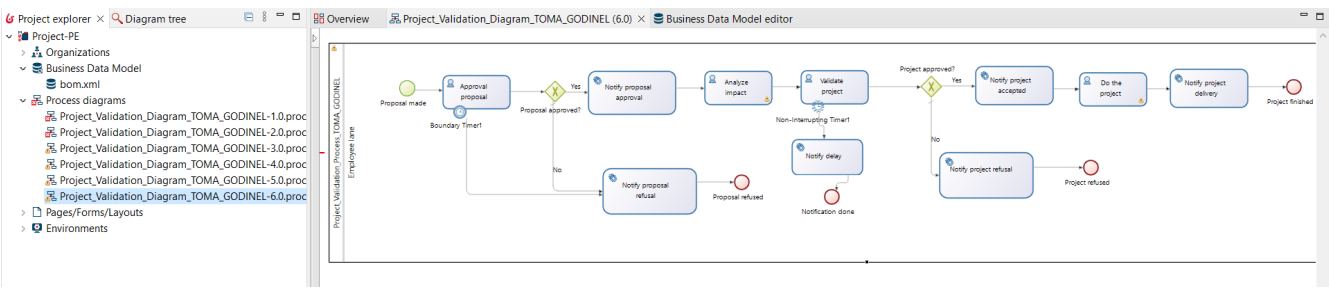
December 10, 2025

1 Tutorial and answers to the questions

1.1 Names of the members of the group

Daria-Teodora GODINEL and Dana-Georgiana TOMA

1.2 Screenshot of the proposed process model including the different copies of the diagrams



In process diagram 6, we implemented the notifications as service tasks and forms for all the human tasks.

1.3 Answers to the six questions of the tutorial

Q1. The model displays some errors. Do you know why?
Answer: The errors occur because the logical gateways lack defined transition conditions. Without these conditions, the process engine cannot determine which of the exclusive paths should be executed.
Q2: Why do we define variables at the process level?
Answer: Variables are defined at the process level to ensure they have global scope. This allows the data to be accessed and utilised by all tasks and events throughout the entire lifecycle of the process instance.
Q3. Why do we use operations in Bonita BPM?
Answer: We use operations in Bonita BPM to define how the data is updated and persisted within the process.
Q4. Which are the human tasks that need a form? Why?
Answer: All human tasks need a for because they need to either input new data required by the system or to view existing information to make a decision.
Q5. What happens if you try to realise the task "Approve proposal" after 15 seconds (suppose you define the timer with 15 seconds)?
Answer: Once the 15 seconds pass, the "Approve proposal" task is cancelled and can no longer be performed. The timer triggers a refusal notification and the process ends for this path.
Q6. Why should we not use a terminate event in this case?
Answer: A terminate event is not used because it would immediately stop the entire process instance. We avoid this to ensure that other parallel flows continue running normally.

2 Summary of the extensions

2.1 Table with a summary of the proposed extensions.

Table 1: Extensions proposed

Extension name	Description	Completeness
Adding actor lanes	Defining distinct lanes to separate the actors in the diagram.	Yes
Email connector	Integrated SMTP connector for notifications with error handling.	Yes
Forms	Enhanced UI forms with read-only data context and validation logic.	Yes
Actors groups	Mapped process lanes to specific ACME organisation groups.	Yes
Do Project Sub-Process	Expanded execution into Frontend, Backend, and Testing phases.	Yes
Analyse Impact Sub-Process	Parallel sub-process for concurrent Technical, Financial and Marketing checks.	Yes
Cancel Project Event	Parallel "Kill Switch" path, allowing cancellation at any execution stage.	Yes
Return Correction Loop	Iterative logic, allowing one proposal modification after refusal.	Yes
Automatic generation of IDs for projects	Scripted generation of unique identifiers for new project instances.	Yes

2.2 General description of critical analysis of the project and difficulties

The implementation phase presented four key technical challenges that required refactoring.

1. **Sub-Process Connectivity:** First attempts to use independent *Call Activities* caused persistent namespace and versioning errors. We resolved this by integrating the workflows as *Embedded Sub-processes* within the main diagram, ensuring runtime stability while maintaining logical separation.

2. **Cross-Context Data Transmission:** Mapping data between disjointed parent and child processes proved complex. Consolidating the sub-processes (as noted above) simplified the variable scope, allowing for direct access to process data and resolving transmission failures.

3. **UI Designer Instability:** Structural modifications to the BPMN diagram frequently severed links to existing forms. To prevent data loss, we had to manually re-map and verify contract inputs and `formOutput` variables after every iteration.

4. **Business Object Initialization:** Moving the initialisation form to a human task initially caused database crashes due to premature `NULL` writes. We fixed this by instantiating a placeholder "Draft" object at the start event, populating the actual attributes (Title, Costs) via setter operations only after the user submits the form.

3 Extension explanation

3.1 Adding actor lanes

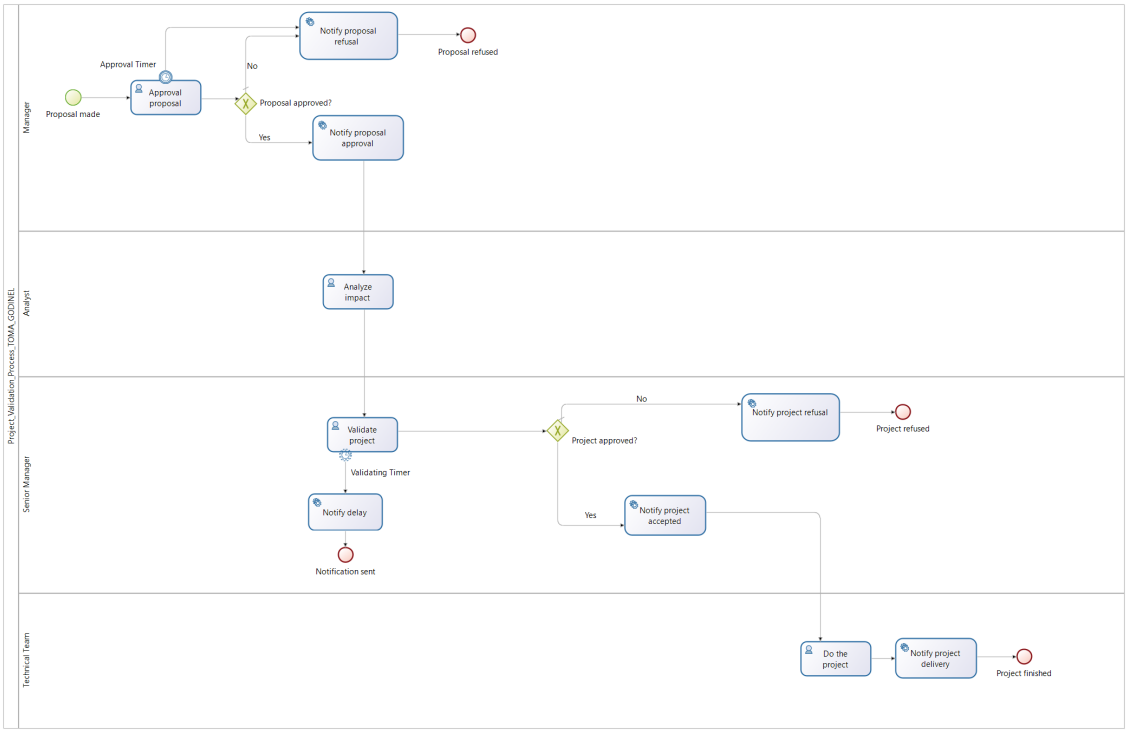


Figure 1: BPMN Diagram of the Extended Project Validation Process with Role Separation

Description of the Extension

We replaced the generic "Employee lane" used in the tutorial with a four-lane structure with different roles. We defined the organisation of the process as follows:

- **Manager Lane:** This lane acts as the process initiator and the first gatekeeper. We assigned the *"Approve proposal"* task here because the initial manager is responsible for filtering requests before they consume technical resources. The automated notifications regarding the proposal's initial status (*Notify proposal approval/refusal*) are also grouped here to reflect the outcome of the manager's decision.
- **Analyst Lane:** We added this lane for the *"Analyse impact"* task. This isolates the impact study from the decision-making steps. By separating this role, we ensure that the impact assessment is populated by a technical expert or business analyst rather than the manager who requested the project.
- **Senior Manager Lane:** The *"Validate project"* task was moved to a Senior Manager lane to distinguish it from the initial approval. This reflects a hierarchy where a higher level of authority is required to grant final budget approval and strategic validation after reviewing the Analyst's report. The automated *"Notify delay"* and final validation notifications are managed within this scope.
- **Technical Team Lane:** The execution phase is isolated in this lane. The *"Do the project"* task is assigned here, ensuring that the actual implementation is allocated to developers or engineers only after all validation steps are complete.

Implementation Details

In Bonita Studio, this was achieved by defining an **Actor** for each lane (Manager, Analyst, Senior Manager and Technical Team) and mapping them to specific groups within the ACME organisation.

3.2 Email connector

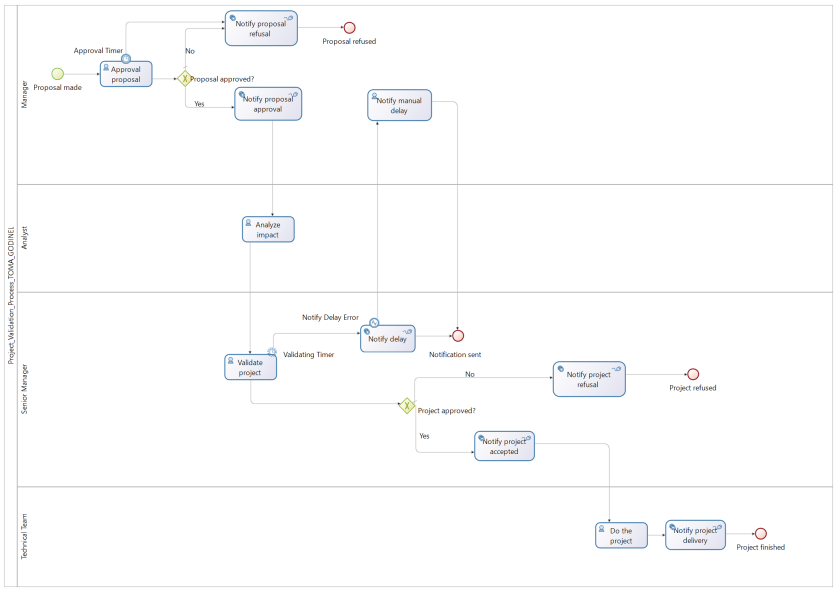


Figure 2: BPMN Diagram of the Extended Project Validation Process with Role Separation

Description of the Extension

We extended all notification tasks in the process (proposal approval, proposal refusal, project validation and delay notifications) with an email connector. To configure each connector, we followed the instructions from the *Tutorial_Extension* document (Sections 6). The email setup uses the fakeSMTP-latest tool provided by the professor on Chamilo, allowing local testing of outgoing messages without requiring an external mail server. Each connector was configured with sender/receiver emails and a message subject reflecting the outcome of the process transition.

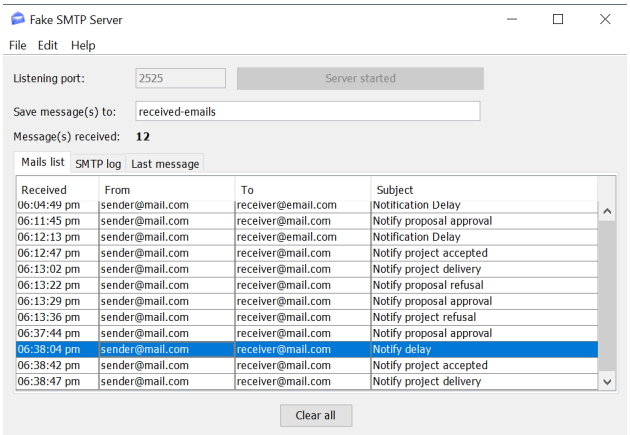


Figure 3: Email Connector Test Result in FakeSMTP

Error Handling

To manage exceptions, we took the *Notify delay task* and we implemented error handling using an interrupting boundary event. A boundary error event was attached directly to the *Notify delay* automatic task and configured to capture any connector-related issues, such as an invalid SMTP configuration or missing connection parameters. When such an error occurs, the event interrupts the automatic task and redirects the flow to a human task, *Notify manual delay*, which allows a user to send the notification manually in case the automated mechanism fails.

3.3 Forms

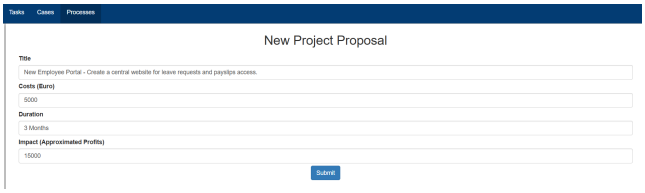


Figure 4: New Project Proposal Form

In Figure 4, the New Project Proposal Form serves as the entry point for the entire workflow (Process Instantiation). It is designed to capture the scope of the project, requiring the requester to input the Title, Estimated Costs, Duration and Projected Profits. Upon submission, these values are mapped to the process contract (‘projectInput‘), which triggers the creation of a new process instance.

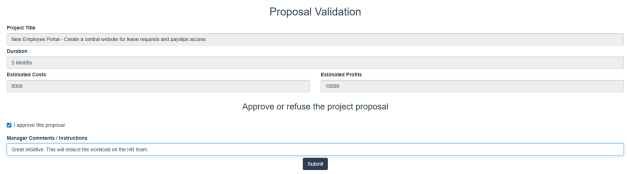


Figure 5: Proposal Validation Form

In Figure 5, the Proposal Validation Form was customised using the UI Designer to replace the default auto-generated interface. To support the Manager’s decision-making process, we implemented a read-only section that retrieves the initial project data (Title, Cost, Proposal) using an External API variable linked to the Business Data Model. The form also includes a decision section where the manager provides an approval status and comments. These inputs are mapped via the process contract to updated attributes in the business object, ensuring instructions are passed to the subsequent ’Analyse Impact’ task.

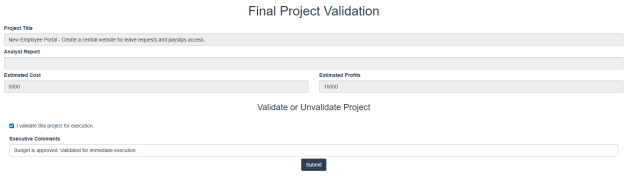


Figure 6: Project Validation Form

In Figure 6, the Project Validation Form is designed as an executive dashboard for the Senior Manager. It receives data from all previous process steps, displaying the initial costs and duration alongside the technical team’s impact analysis in a read-only format. This design ensures that the final Go/No-Go decision is data-driven. The form records the validation status and executive directives, which determine whether the process proceeds to the ’Do Project’ phase or terminates.

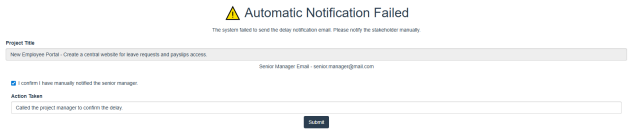


Figure 7: Manual Notification of Delay Form

In Figure 7, this form is a component of the error management extension. It is triggered specifically when the automatic email connector fails (caught by a boundary error event). The interface is designed to alert the administrator immediately with a clear warning and provide the necessary project context to assist manual intervention. It requires a confirmation checkbox before the case can be closed.

3.4 Actors groups

To ensure proper role-based task execution, we extended the process by assigning dedicated actors to distinct organisational groups that correspond to the lanes in the BPMN model. For each lane (Manager, Analyst, Senior Manager and Technical Team), we created a separate group in the Bonita organization structure (Fig. 8) and added a user whose name reflects the lane for clarity and ease of testing (Fig. 9). This grouping structure mirrors the division of responsibilities within the process and establishes a clear mapping between organizational entities and their corresponding process roles.

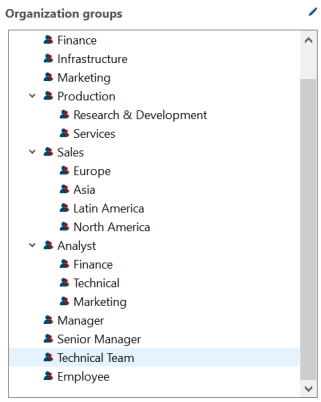


Figure 8: New groups in the Bonita organisation

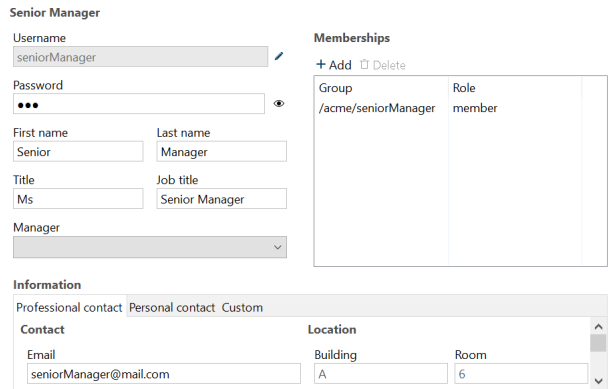


Figure 9: Example of a user created: Senior Manager

After creating the groups and users, we defined new Actors at the pool level (Fig. 10) and assigned each lane to its appropriate Actor (Fig. 11). We then configured the Actor Mapping to associate each Actor with the corresponding group we had created (Fig. 12). This configuration ensures that only members of the designated group can perform the tasks located within their assigned lane. As a result, the process now enforces correct responsibility boundaries, preventing unauthorised users from executing tasks and improving the security of the workflow.

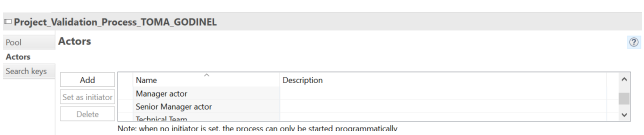


Figure 10: New actors defined in the pool

Figure 11: Actor assigned in a lane

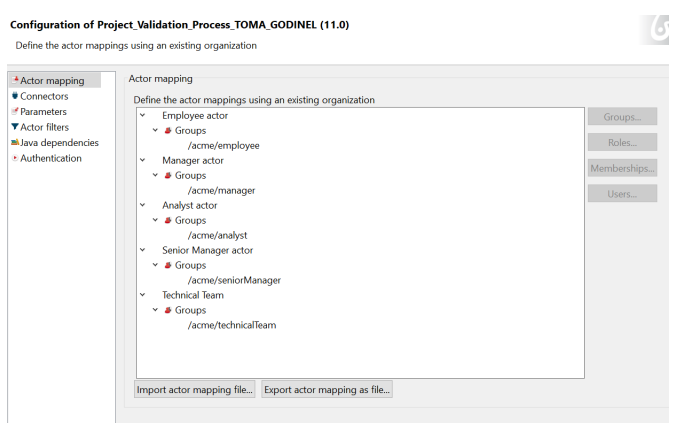


Figure 12: Mapping between actors and groups

3.5 Do Project Sub-Process

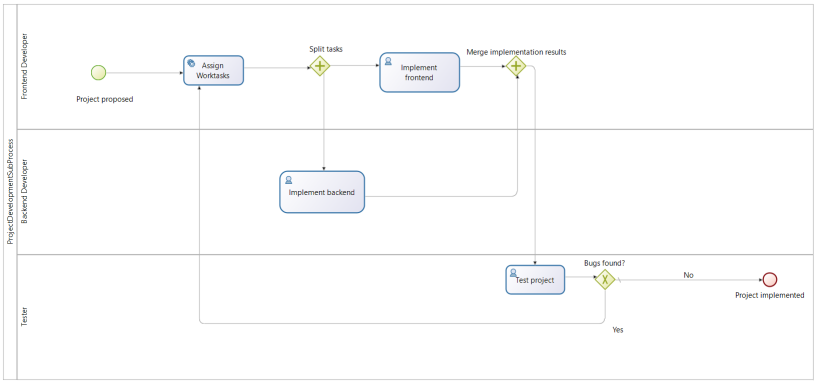


Figure 13: BPMN Diagram of the Project Development Sub-process

We extended the project by introducing a *Call Activity* that triggers a dedicated sub-process for the *Do the project* phase. We attempted to implement this sub-process in a separate diagram, but we were unable to connect it to the main process. As a result, we decided to include the sub-process within the same diagram. This sub-process models the internal workflow of a technical team, dividing the work among three actors: Front-End Developer, Back-End Developer and Tester. The sub-process begins with an automated task, *Assign worktasks*, which simulates the automatic distribution of tasks to the front-end and back-end roles. This task leads into an *AND* gateway, creating two parallel execution paths where the front-end and back-end developers independently complete their implementation tasks. Once both branches finish, the flow moves to the Tester, who evaluates whether the implementation contains bugs. If bugs are detected, the process loops back to the task assignment step, allowing the team to iterate on fixes. If no bugs are found, the sub-process terminates and the main process resumes. All tasks within the sub-process use functional forms to ensure accurate data input, user interaction and variable value transmission back to the parent process.

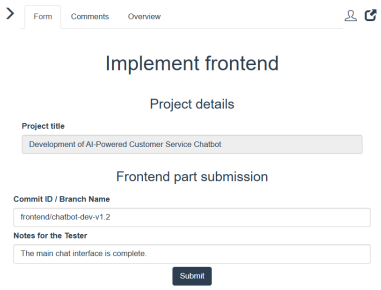


Figure 14: Frontend Developer Form

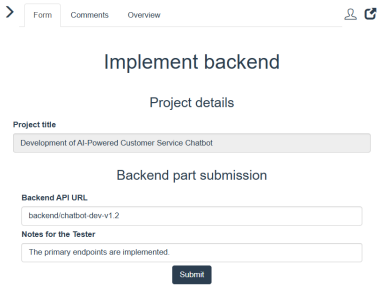


Figure 15: Backend Form

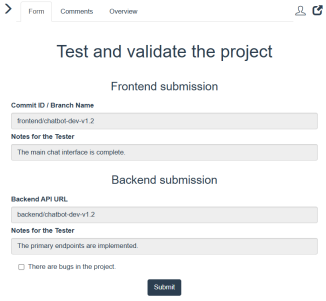


Figure 16: Tester Form

3.6 Analyse Impact Sub-Process

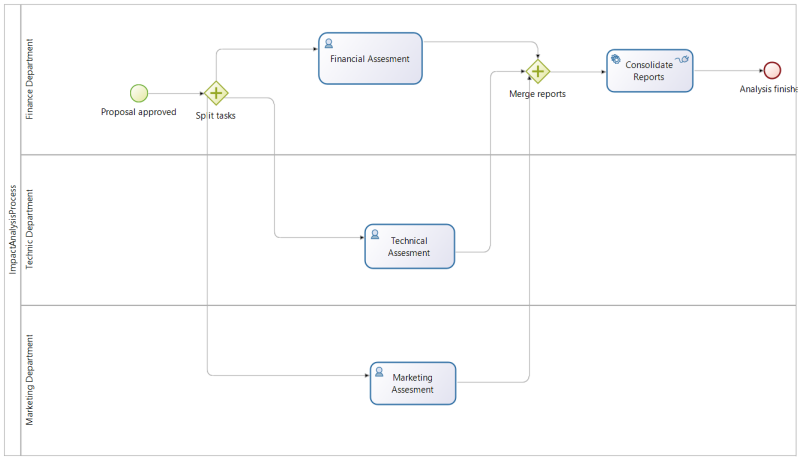


Figure 17: BPMN Diagram of the Analyse Impact Sub-Process

We extended the project by adding a *Call Activity* that launches a dedicated sub-process for the *Analyse impact* task. This sub-process models a multi-department evaluation, involving three distinct actors: the Finance Department, the Technical Department and the Marketing Department. The sub-process begins with an *AND* gateway, which splits the workflow into three parallel branches so that each department can independently perform its analysis and produce its respective report. Once all three analyses are completed, the flows continue with an automated task that compiles the collected reports into a consolidated document. This automated task then forwards the combined results to the Senior Manager for validation. After this, the sub-process ends and control returns to the main process, which continues with the validation step of the senior manager.

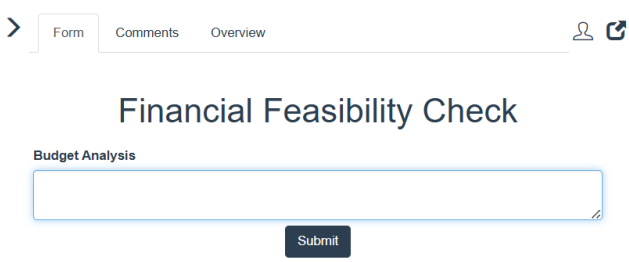


Figure 18: Finance Analyst Form

Figure 19: Technical Analyst Form

Figure 20: Marketing Analyst Form

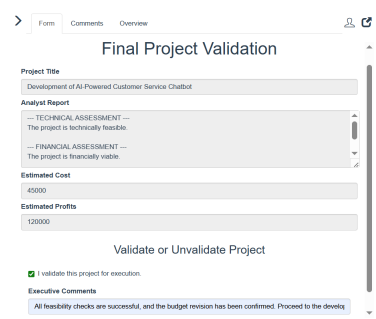


Figure 21: Senior Management Form

3.7 Cancel Project Event

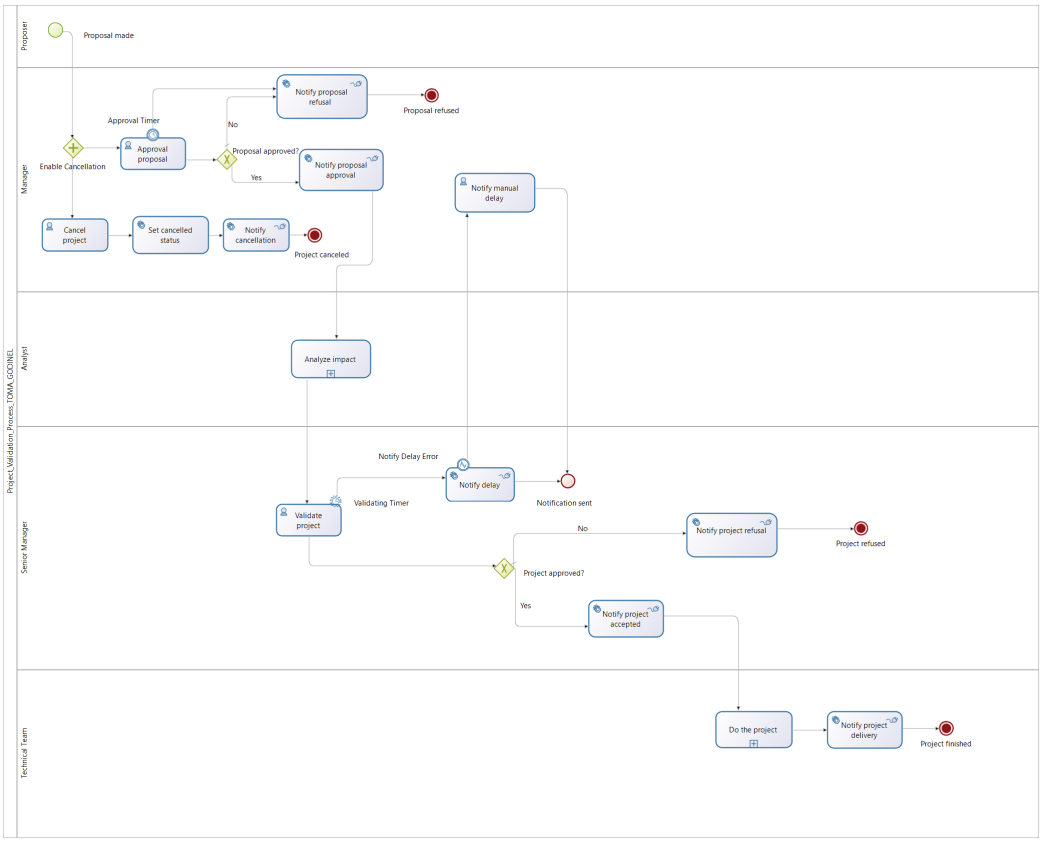


Figure 22: BPMN Diagram of the main process with the cancellation task

To distinguish the process initiator from the approval authority, we added a dedicated Proposer Lane at the top of the diagram to contain the start event and initial submission tasks.

To allow project cancellation at any time, we placed a *Parallel Gateway* right after the start event. This splits the workflow into two simultaneous paths: the normal project cycle and a separate cancellation path.

The cancellation path adds a permanent *Cancel project* task for the Manager. If used, this task updates the database status and sends a notification.

To make this work correctly, we replaced all End Events with *Terminate End Events*. This ensures that finishing one path automatically stops the other. For example, if the project is successfully delivered, the "Cancel" task disappears. Conversely, if the Manager cancels the project, any active validation tasks stop immediately, preventing stuck tasks in the system.

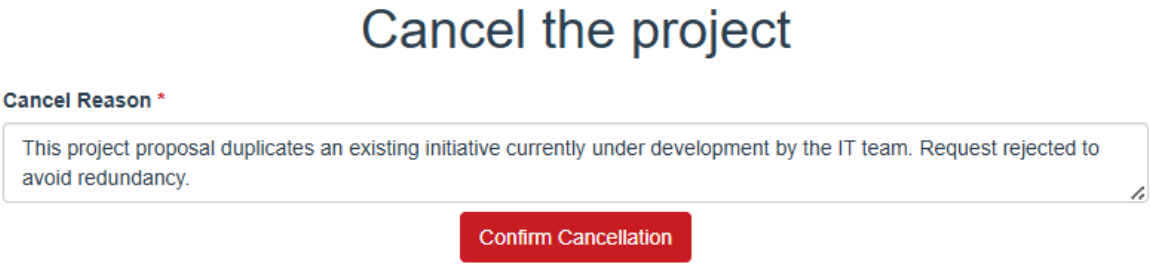


Figure 23: Cancel Project Form

3.8 Return Correction Loop

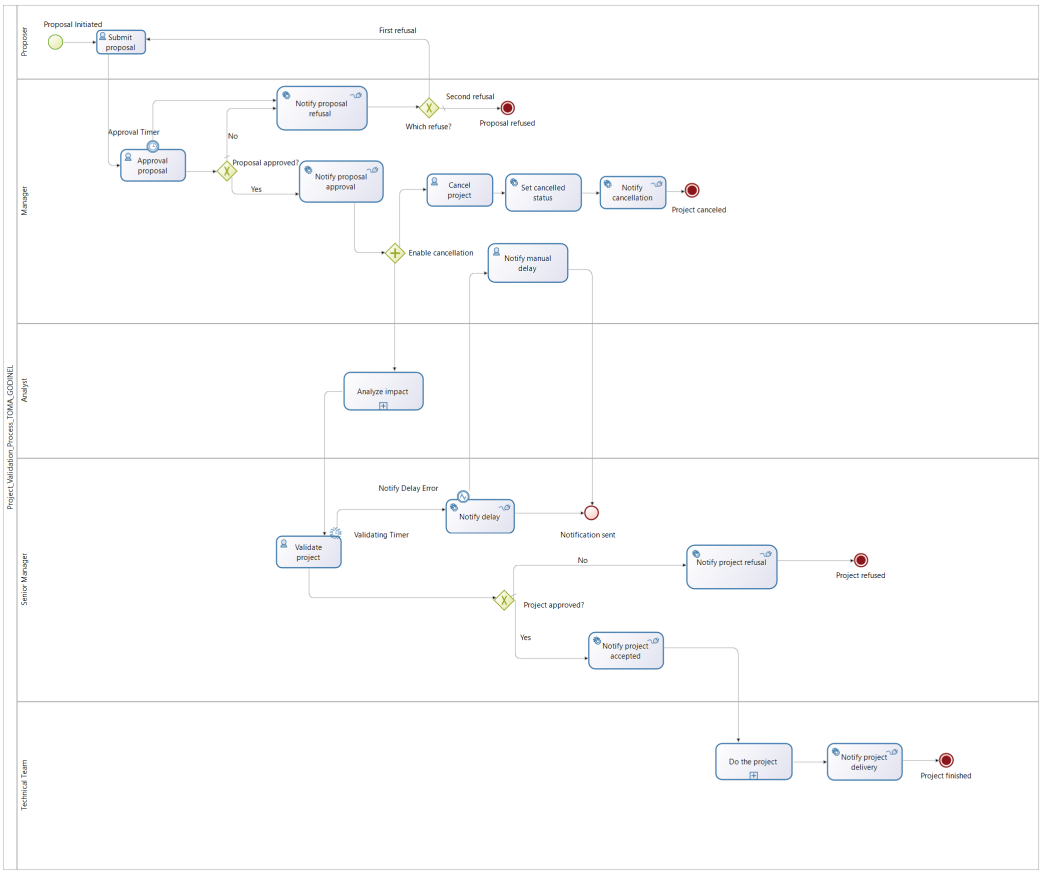


Figure 24: BPMN Diagram of the main process with the return loop

To avoid immediate rejection, we added a "correction loop". If the manager refuses the proposal, the process does not end immediately, instead, it returns to the Proposer. We use a variable (‘refusalCount‘) to track these attempts. If the manager refuses a second time, the process ends permanently.

We moved the initial form from the Start Event to the new "Submit Proposal" task. This allows the same form to be used for both the initial submission and any necessary modifications later.

Integration with Cancellation Logic: We moved the Parallel Gateway ("Enable Cancellation") so it triggers only after the proposal is approved. This separates the process into two stages:

- *Drafting Phase:* The proposal can be modified and resubmitted without creating unnecessary "Cancel" tasks.
- *Execution Phase:* Once approved, the gateway starts the next step and simultaneously enables the "Cancel Project" task.

We used Terminate End Events to ensure that cancelling the project stops all other active tasks immediately.

3.9 Automatic generation of IDs for projects

Figure 25: Business Data of a case with projectID

We extended the project by adding a process variable named *projectID* to uniquely identify each project instance. This variable is automatically generated at the start of the process and follows the format *PROJ-year-processId*, where year corresponds to the current year and processId is the unique identifier assigned by Bonita for the process instance. The *projectId* variable is stored as a process variable and is available throughout the process, allowing all tasks, forms and subprocesses to reference the same unique identifier for consistency and traceability.

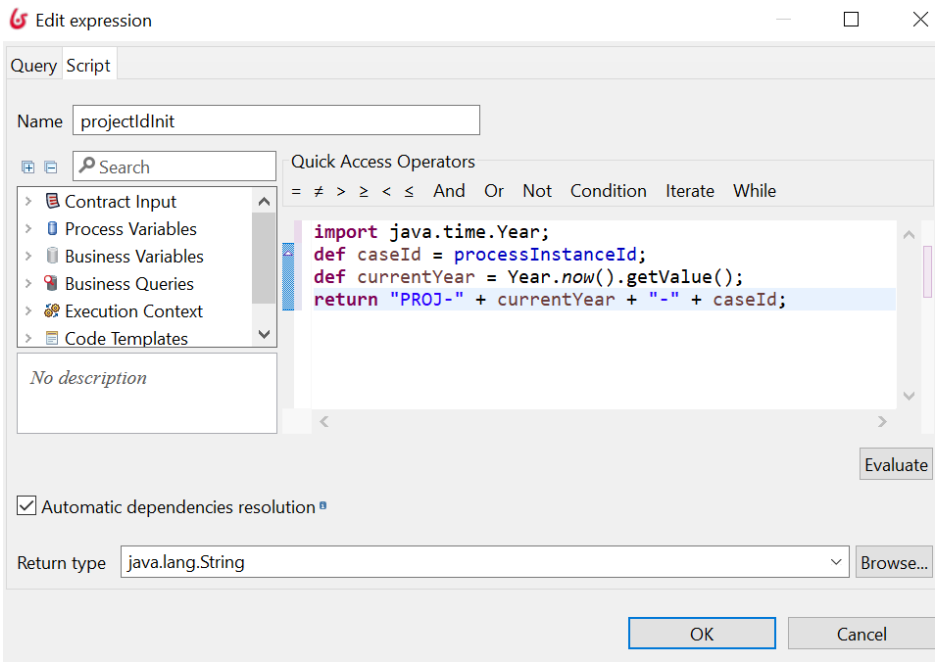


Figure 26: Script for initialising the projectID

We implemented the *projectID* using a Groovy script. The script first retrieves the current process instance ID from *processInstanceId* and the current year using *Year.now().getValue()* from Java's *java.time.Year* class. It then combines these values into a formatted string in the form *PROJ-year-processId* (for example, PROJ-2025-1503).